ECHO

Tour de Code

Version v0.1-draft
Commit: HEAD
January 19, 2026

James Ross & Echo Contributors

Copyright

Copyright (c) James Ross and FLYING ROBOTS; Echo contributors. Licensed under Apache-2.0 OR MIND-UCAL-1.0.

Trademarks

Echo and associated marks may be trademarks of their respective owners.

Warranty

Provided “as is”, without warranties or conditions of any kind.

Source

This booklet is generated from the Echo repository documentation.

Foreword

Echo is a deterministic, multiverse-aware engine. This booklet walks you in with progressive layers: orient yourself, learn the core building blocks, then dive into math and operations. Each shelf can stand alone; together they form the full Echo field guide.

If you are new, start with the onboarding roadmap and glossary. If you build or extend Echo, keep the determinism contract and scheduler flow in view. Future work will deepen each part and add more diagrams as Echo evolves.

Contents

Part I

Orientation

Chapter 1

Onboarding Roadmap (First 30–60 Minutes)

1.1 Get the Toolchain

- Install Rust per `rust-toolchain.toml` (workspace MSRV).
- Install Node ≥ 18 and `pnpm` (see `packageManager` in `package.json`).
- Run `make hooks` once to enable repo checks.

1.2 Clone, Smoke Test, and Guard Rails

- **Clone**: `git clone` and `pnpm install` (docs) / `cargo fetch` (Rust).
- **Docs dev**: `pnpm docs:dev` to confirm VitePress works.
- **Rust sanity**: `cargo test -p warp-core - -list` to ensure toolchain is healthy.
- **Hooks**: verify pre-commit runs (fmt/clippy/docs guard) by committing a no-op change locally.

1.3 Read This First

1. **Orientation**: `architecture-outline.md` (why Echo exists).
2. **Determinism contract**: skim `determinism-invariants.md` for guarantees.
3. **Branching model**: `branch-merge-playbook.md` (merge-only; no rebases/force pushes).
4. **Work map**: GitHub Issues and project board for current tasks.

1.4 Do One Hands-On Task

- Run a focused test: `cargo test radix_drain -package warp-core`.
- Explore a doc: open `docs/public/collision-dpo-tour.html` to see visuals.

1.5 Next Hour: Choose Your Track

- **Runtime**: read Scheduler + Storage specs, then run scheduler benchmarks.
- **Docs/Math**: read deterministic math specs; reproduce a figure in TikZ.
- **Ops**: rehearse the merge playbook and run Docs Guard locally.

Chapter 2

Glossary

Echo Deterministic, multiverse-aware ECS runtime.

Chronos / Kairos / Aion Time lenses: real-time execution (Chronos), branching timelines and merges (Kairos), and meta-time/agency (Aion).

Codex's Baby Echo's scheduler core that orders and drains work deterministically.

Branch Tree Structure tracking divergent timelines and merges; governed by the merge playbook.

Determinism Contract Guarantees covering hashing, scheduling order, canonical floats, and replayability.

Dirty Index Tracking of modified components for efficient archetype updates.

Scheduler Kind Configurable scheduler implementation (Radix default; Legacy/BTree fallback).

Temporal Bridge Mechanism for retro/forward delivery without paradox; works with paradox guard.

Snapshot / Commit Canonical state plus provenance (parents, digests) used for replay and verification.

Docs Guard Policy that code changes must update docs (execution plan).

2.1 What is Echo?

Echo is a **deterministic, multiverse-aware Entity Component System (ECS)** engine. Unlike traditional game engines that prioritize visual fidelity or editor convenience, Echo prioritizes **determinism**, **state preservation**, and **simulation integrity**.

It is built on the concepts described by the Foundations Series papers—"Worldline Algebra for Recursive Provenance" (WARP) graphs—meaning that the entire game state is a WARP graph, and updates are deterministic graph rewrites. This allows for features that are practically impossible in standard engines, such as:

- **Perfect Determinism:** Given the same initial state and inputs, the simulation will evolve identically on any machine (x86, ARM, WASM).

- **Time Travel:** The ability to rollback the simulation to any previous tick, branch time into alternative realities, and merge them back.
- **Renderer Agnosticism:** Echo does not care how it is drawn. It emits an event stream that describes the latest graph writes applied to the State at a given tick.

2.1.1 Core Philosophy

Echo was born from earlier prototypes and operates under a set of strict cultural principles:

1. **Tests Are The Spec:** Documentation as performance art.
2. **Hexagonal Domain Boundary:** Pattern of patterns.
3. **We Do Not Rewrite History:** Reminder: The immutable ledger makes git blame eternal.
4. **Predictable Loop:** Every frame has a cryptographic description.

2.1.2 Who is Echo For?

Echo is designed for general use. It uses WARP graphs, which give it:

- **RTS Games:** Where thousands of units must remain in sync across a network.
- **Fighting Games:** Where rollback netcode (GGPO-style) is a requirement, not a feature.
- **Scientific Simulations:** Where reproducibility is critical.
- **Collaborative Tools:** Where multiple users edit a shared state (using the branching/merging capabilities).

2.2 Echo vs. The World

Echo differs significantly from general-purpose engines like Unity, Unreal, or Godot. While those engines optimize for "getting something on screen fast" and providing a rich editor, Echo optimizes for **correctness under complex temporal manipulation**.

2.2.1 Feature Comparison Matrix

2.2.2 Key Differentiators

1. Determinism as a Constraint

In Unity or Unreal, ‘float‘ math depends on the CPU architecture. A simulation running on an Intel chip might diverge from one on an ARM chip after a few thousand frames. Echo uses `F32Scalar` (canonicalized floats) or fixed-point math to ensure bit-perfect identity across all platforms. This enables:

- **Replay Systems:** Store only inputs, replay the game exactly.

Feature	**Echo**	**Unity/Unreal**	**Godot**
Architecture	Pure ECS (hexagonal ports/adapters)	GameObject / Actor	Node Tree
Determinism	**Core constraint**; radix or legacy scheduler, stable order	Best effort / plug-in	Best effort
State Model	Immutable + COW snapshots	Mutable in-place	Mutable in-place
Netcode	Native rollback/replay (input or state sync)	State sync (mostly)	RPC / state sync
Branching	Native multiverse timelines	Custom / atypical	Not supported
Math	Canonical zero float wrapper; fixed-point planned	Hardware float	Hardware float
Rendering	Decoupled port; adapters pick GPU/OS	Tightly coupled	Tightly coupled

Table 2.1: Comparison of core behaviors (emphasis on determinism and branching).

- **Anti-Cheat:** Verify game logic on the server with zero tolerance for deviation.
- **Desync Debugging:** If a bug happens, it happens exactly the same way every time.

2. The Multiverse (Branching)

Traditional engines have one "World". To do a "prediction" (e.g., for netcode), they must serialize the whole world, step forward, and then deserialize it back (a slow process). Echo uses **Copy-on-Write (COW)** archetype storage. It can fork the world instantly.

- **Main Timeline:** The "real" game.
- **Speculative Branch:** Used by AI to "think ahead" or by netcode to predict movement.

These branches share memory for unchanged data, making branching extremely cheap.

3. Rulial Multi-Graph vs. Object Update

In typical engines, objects have an ‘Update()‘ function. Order of execution is often ill-defined or rigid. Echo uses a graph rewriting approach. Systems define rules. The Scheduler identifies independent subgraphs that can be updated in parallel (using MWMR - Multi-Writer Multi-Reader patterns) without race conditions, because the data dependency is explicit.

Part II

Core Concepts

Chapter 3

High-Level Architecture

Echo follows a strict **Hexagonal Architecture** (or Ports and Adapters pattern). The core domain logic sits in the center, isolated from the outside world.

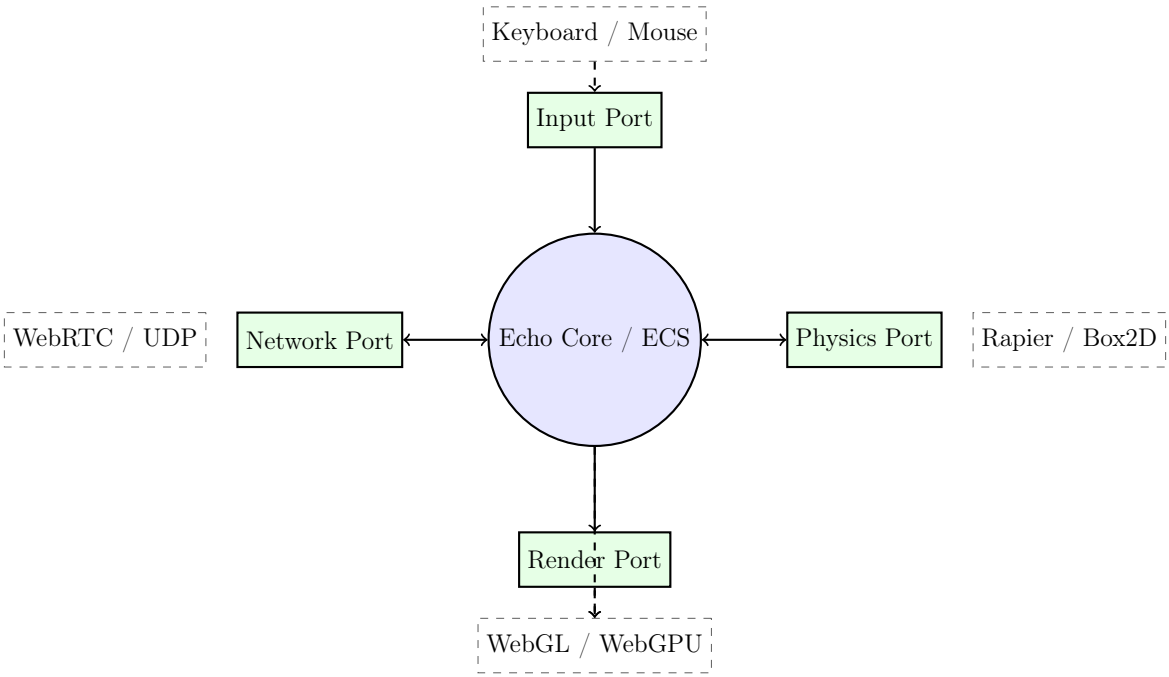

Figure 3.1: Echo's Hexagonal Architecture

3.1 Domain Layers

The architecture is stratified into clear layers:

1. **Core ECS:** The heart of the engine.
    - **Entities:** Numerical IDs managed by a high-watermark allocator.
    - **Components:** Data buckets. Storage is archetype-based (chunks sized for CPU cache lines) with Copy-on-Write (COW) support for branching.
    - **Systems:** Pure functions that transform data.

2. **World & Scene Management:** Handles the lifecycle of entities and the "Scene Graph".
3. **Time & Simulation (Timecube):** Manages the temporal axes:
   - **Chronos:** The monotonic tick counter (Sequence).
   - **Kairos:** The branch identifier (Possibility).
   - **Aion:** The narrative weight/entropy (Significance).
4. **MaterializationBus:** A deterministic event bus used for communication between systems and for bridging the gap between the Core and the Ports. See ADR-0003 (MaterializationBus).

3.2 Ports & Adapters

Echo does not define *how* things happen, only *that* they happen.

- **Renderer Port:** Receives a ‘FramePacket‘ containing generic draw commands (mesh refs, transforms). Adapters (e.g., PixiJS, wgpu) translate this to GPU calls.
- **Input Port:** Aggregates input into snapshots. The domain polls this once per frame.
- **Physics Port:** Dual-write system. ECS components store the "desired" state; the Physics Port (via an adapter like Rapier) advances the simulation and syncs the authoritative transform back.
- **Networking Port:** Handles replication. Thanks to determinism, it can use state-snapshots or input-replication (GGPO).

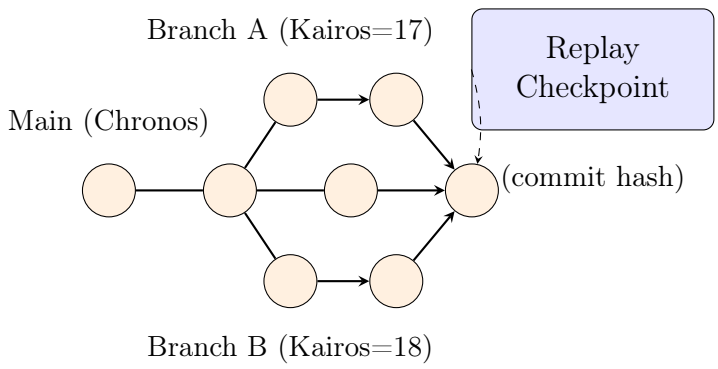

Figure 3.2: Git-style branching: Chronos is the main line, Kairos branches diverge and deterministically merge;
checkpoints use snapshot hashes (Aion weighting optional).

3.2.1 Deterministic Scheduler & Sandbox

- **Scheduler kinds:** ‘SchedulerKind‘ lets the engine pick the default Radix scheduler (stable $O(n)$ LSD radix on `scope_hash`, `rule_id`, `nonce`) or a Legacy BTreeMap path for A/B runs.
- **Footprint conflicts:** Independence checks use generation-stamped sets over node/edge read-write sets and boundary ports (MWMR). Any overlap conflicts and the rewrite aborts before execution.
- **Sandbox A/B:** `EchoConfig` + `run_pair_determinism` build two isolated engines (different schedulers, seeds, or rule sets) and compare snapshot hashes step-by-step to prove determinism.
- **Snapshots:** Each tick can emit a `Snapshot` with a 32-byte BLAKE3 hash used for comparison, replay, or merge validation.

Chapter 4

Timeline Anatomy (Kairos Lens)

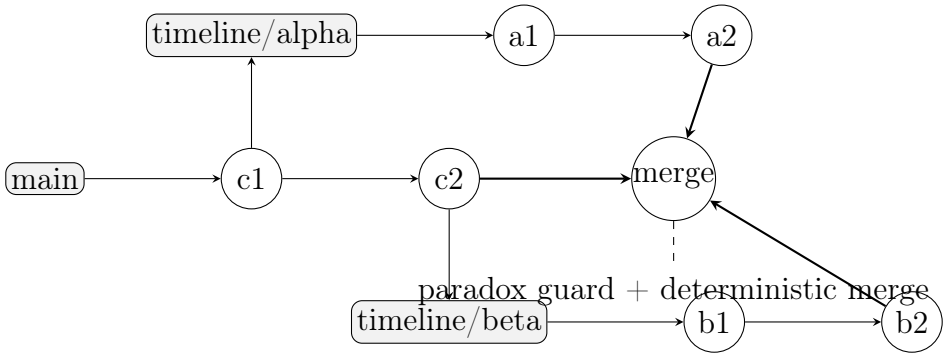

Figure 4.1: Branching timelines converge through deterministic merge + paradox guard.

Read this diagram as: main spawns experimental timelines; each accumulates commits, then merges through the paradox guard. All merges are deterministic and recorded in the branch tree for replay.

Chapter 5

Deterministic Scheduler Flow

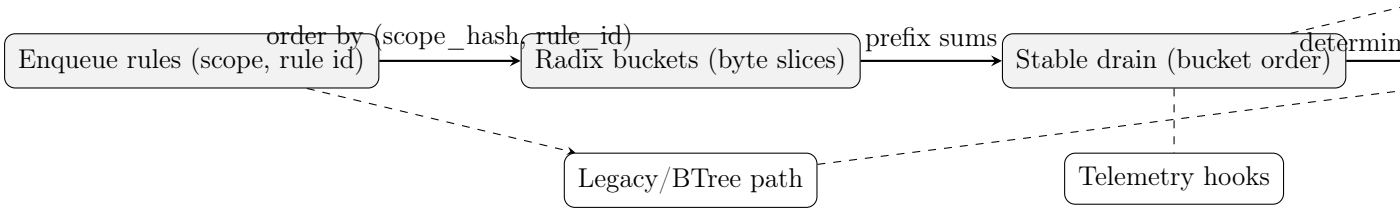

Figure 5.1: Codex's Baby (radix scheduler) keeps ordering stable via bucketed drains; conflicts are resolved deterministically before commits.

Key invariants:

- Ordering is stable: sorted by $(scope_hash, rule_id)$; radix buckets preserve order inside each byte slice.
- Footprint conflicts fail fast; no silent drops.
- Legacy/BTree path is optional for side-by-side comparisons.
- Telemetry and paradox guard hook in without altering order.

Chapter 6

ECS Storage Layout

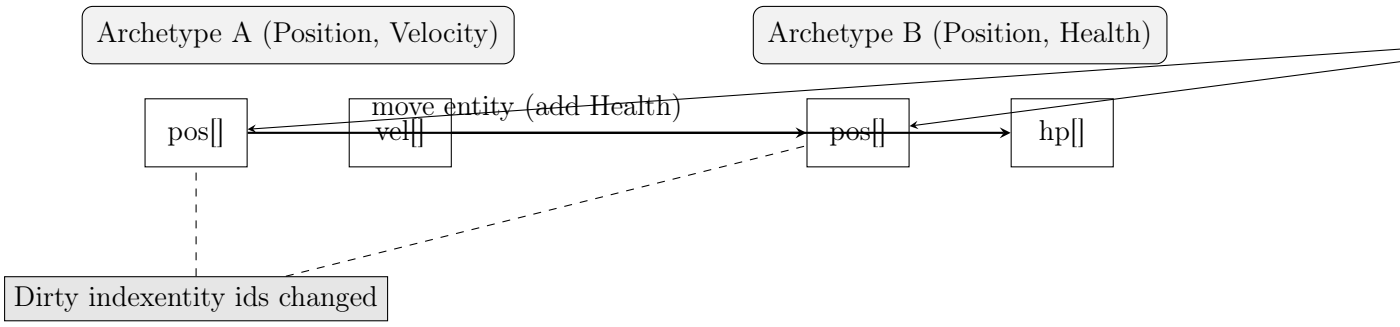

Figure 6.1: Archetypes store columnar component arrays; moving entities rewrites sparse maps
and bumps the dirty index for deterministic diffs.

Notes:

- Archetypes are columnar; moves are deterministic rewrites, not lazy copies.
- Dirty index tracks touched entities for fast diffing and replay.
- Sparse maps give O(1) entity → row lookup; kept in sync on every move.

Chapter 7

BOAW: Best of All Worlds Storage

The BOAW architecture replaces the spike-era monolithic in-place mutation model with immutable snapshots, COW overlays, and lockless parallel execution. The result: deterministic convergence across platforms, massive parallel throughput without locks, and zero-copy snapshot IO.

7.1 The BOAW Stack

1. **Immutable Base** — Snapshots are immutable and read-only (WSC-backed; optionally memory-mapped).
2. **TickDelta Overlay** — Rewrites do not mutate a shared store. They emit ops into thread-local deltas.
3. **Deterministic Admission** — Only independent rewrites are admitted per tick (footprints). Ordering is canonical.
4. **Lockless Parallel Execute** — Workers execute admitted rewrites against a read-only snapshot, emitting ops locally.
5. **Deterministic Merge-of-Deltas** — Thread-local deltas are merged canonically into a single tick patch.
6. **Canonical Commit** — Apply patch to build the next snapshot, compute state root + patch digest + commit hash.
7. **Collapse/Merge** — Multi-parent merges reconcile claims and structured state, with typed merge rules.
8. **Privacy** — In mind mode, the ledger records only commitments/proofs—never sensitive bytes.

7.2 Key Concepts

Snapshot (WSC) Immutable, canonical, readable without copying: sorted tables + ranges + blob arena.

COW "Delete" means unlink from the worldline view, not physical destruction; physical reclamation is optional GC.

Footprint Declared read/write sets (nodes, edges, attachments, boundary ports) used for independence checks.

TickDelta Thread-local append-only ops produced during execution.

Mind mode Ledger must be publishable; secrets never enter provenance.

7.3 COW Mechanics

During a tick, nothing shared is mutated. Writes produce an overlay delta. The next commit produces a new immutable snapshot that structurally shares with prior snapshots.

- Base snapshot is immutable
- Overlay is per-worldline/per-tick
- Commit materializes a new snapshot (sharing unchanged segments)
- Delete semantics are "Unlink"—absent from next snapshot's reachable-only view

7.4 Tick Pipeline

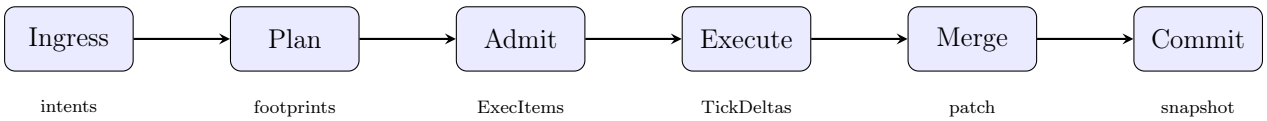

Figure 7.1: BOAW tick pipeline: ingress → plan → admit → parallel execute → merge → commit.

7.4.1 Canonical Ingress

Ingest intents into an ingress list keyed by canonical `intent_id`. Sort deterministically (radix sort permitted if stable and defined).

7.4.2 Plan

For each candidate intent/rule match: compute footprint, compute `factor_mask`, produce a `PlannedRewrite` handle that reads from snapshot only.

7.4.3 Admit (Deterministic)

Greedy admit in canonical order by `intent_id` (and tie-breakers like `rule_id`, `match_ix`). Reject or defer conflicts deterministically.

7.4.4 Execute (Parallel, Lockless)

Workers execute admitted rewrites against read-only snapshot. Output is a thread-local `TickDelta` (append-only ops).

7.4.5 Merge Deltas (Deterministic)

Concatenate all thread-local ops. Sort canonically by `(op_kind, object_key, tie_breaker)` where tie-breaker is stable (`intent_id / rule_id / match_ix`). Apply per-type conflict policies.

7.4.6 Commit

Build next reachable-only snapshot from previous snapshot + merged ops. Compute:

- `state_root` — canonical hash of materialized reachable state
- `patch_digest` — canonical hash of merged ops
- `commit_hash` $= H(\text{parents} \parallel \text{state_root} \parallel \text{patch_digest} \parallel \text{schema} \parallel \text{tick} \parallel \text{policy})$

7.5 Free Money Invariance

The "free money" property: workers $\in \{1, 2, 4, 8, 16, 32\}$, any ingress permutation—`patch_digest`, `state_root`, `commit_hash` are identical.

This is the determinism drill sergeant for BOAW. If this invariant fails, a footprint bug exists.

7.6 Virtual Shards

We explicitly reject "queue per CPU" as the partition key because it is hardware-dependent. Instead:

- Fixed virtual shards (e.g., 256, power-of-two)
- Route by existing `NodeId`/`EdgeId` bits: `shard = lowbits(id) & (SHARDS-1)`
- Work queues are per shard, not per core
- Workers pull/claim shards dynamically to balance load

This preserves determinism across machines and improves locality.

7.7 Doctrine

> *State is an immutable snapshot. Time is a commit DAG. Writes are deltas. Deletes are unlinks. Proofs are claims. Privacy is non-negotiable. Determinism isn't optional.*

Chapter 8

The Game Loop

Echo's game loop is a rigid, phase-based pipeline designed to ensure causality and data consistency.

8.1 The Loop Phases

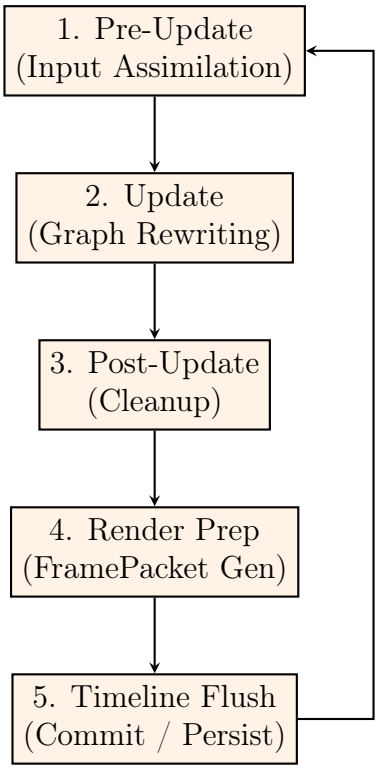

Figure 8.1: Echo's five-phase deterministic game loop.

1. **Pre-Update:**
   - Inputs from the 'InputPort' are polled and injected into the 'InputComponent'.
   - The `MaterializationBus` flushes pending external events.
2. **Update (The Core):**

- The Scheduler executes Systems.
- Physics simulation steps (if sub-stepping is enabled).
- Game logic runs.

3. **Post-Update:**
   - Late-frame logic (e.g., camera smoothing).
   - Entity despawning cleanup.
4. **Render Prep:**
   - Systems read the state and produce a ‘FramePacket‘.
   - No state mutation is allowed here.
5. **Timeline Flush:**
   - The diffs for the frame are finalized.
   - The "State Root" hash is computed.
   - The frame is committed to the Timeline (Chronos).

8.2 Graph Rewriting (The Scheduler)

Inside the **Update** phase, Echo does not just "loop over arrays." It performs **Graph Rewriting**.

1. **Matching:** Systems query the graph for patterns (e.g., "Entity with Position and Velocity").
2. **Reserving:** The Scheduler identifies which matches are independent (don’t touch the same data).
3. **Execution:** Independent matches are processed. They produce a "Rewrite" (a list of changes).
4. **Application:** The Rewrites are applied to the graph state.

This "Reserve-Execute-Apply" cycle ensures that even in a parallel execution environment, the result is deterministic. If two systems try to modify the same entity, the Scheduler detects the conflict and orders them deterministically based on a topological sort of the System Graph.

8.3 Low-Level Networking: JS-ABI Wire Protocol

Echo’s engine and tools communicate over a small, deterministic wire protocol called *JS-ABI v1.0*. This section describes the on-the-wire framing: how bytes are grouped into packets, how integrity is checked, and how the CBOR payloads (OpEnvelopes) fit into the story.

Detailed rationale, test vectors, and the deterministic encoding profile live in the JS-ABI architecture documents (ADR-0013 and ARCH-0013); here we focus on the subset

needed to reason about Echo's engine, session service, and tools. The way the session service uses this framing is described in Section **??** and in more detail for tools in Section **??** (Session Service) and Section **??** (WARP Viewer) in the Editor Tools volume.

8.3.1 Packet Framing Overview

Every JS-ABI packet has three parts:

- A fixed-size **header** (12 bytes).
- A variable-size **payload** (Canonical CBOR OpEnvelope).
- A fixed-size **checksum** (32 bytes, BLAKE3-256).

Formally:

$$\text{PACKET} = \text{HEADER} \parallel \text{PAYLOAD} \parallel \text{CHECKSUM}.$$

All multi-byte integers in the header are big-endian (network byte order).

8.3.2 Header Fields

Field	Size (bytes)	Description
`MAGIC`	4	Protocol signature (`0x4a495421` = "JIT!")
`VERSION`	2	Wire protocol version (`0x0001` for v1.0)
`FLAGS`	2	Reserved feature bits (compression, streaming, ...)
`LENGTH`	4	Length of the payload in bytes (N)
Total	**12**	

Table 8.1: JS-ABI v1.0 header layout.

The payload immediately follows the header and contains a single CBOR `OpEnvelope` (Section **??**); the checksum covers both header and payload.

8.3.3 OpEnvelope Payloads

The payload is always a deterministically encoded Canonical CBOR map with the following logical shape:

```
op-envelope = {
  "op": tstr,   ; operation name (e.g. "handshake", "error", "warp_stream")
  "ts": uint,   ; logical timestamp (kernel clock, authoritative on server)
  "payload": any
}
```

The `op` field selects a concrete payload schema (handshake, error, WARP stream, ...). The `ts` field is the kernel logical timestamp; the server is the sole authority for WAL-participating timestamps.

Canonical CBOR rules (definite lengths only, preferred integer encodings, canonical map ordering, no tags) are enforced at the encoder and decoder. Any deviation is treated as non-compliant for WAL, hashing, or deterministic replay.

8.3.4 End-to-End Byte Layout (TikZ Overview)

Figure **??** sketches a single packet on the wire.

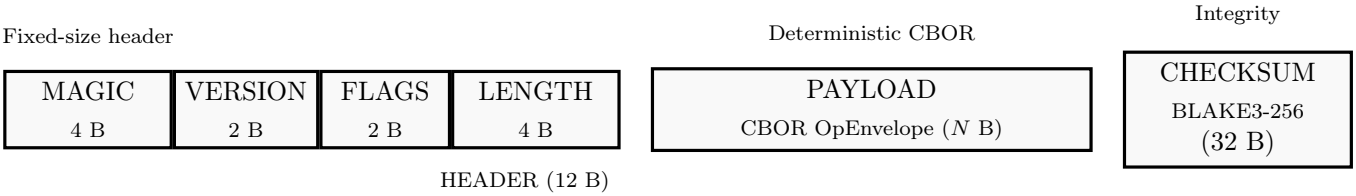

Figure 8.2: JS-ABI v1.0 packet layout: header + payload + checksum.

8.3.5 Parsing and Verification Flow

A compliant implementation follows a fixed sequence when reading from a byte stream:

1. **Read header** (12 bytes) and validate:
   - `MAGIC` matches the expected signature.
   - `VERSION` is supported.
   - `FLAGS` are either zero or a negotiated subset.
2. **Read payload** of length `LENGTH` into a buffer.
3. **Read checksum** (32 bytes).
4. **Compute BLAKE3-256** over HEADER ‖ PAYLOAD.
5. **Compare** the result with the received checksum: mismatch ⇒ reject the packet (data corruption).
6. **Decode CBOR** payload into `OpEnvelope`, enforcing the Canonical CBOR subset.

Only packets that pass all of these checks are admitted into the kernel (for write-ahead logging and deterministic replay). Tools that sit "at the edge" of the system may provide more lenient modes for debugging, but those modes are outside JS-ABI v1.0 compliance.

8.4 Consuming WARP Streams: Snapshots and Diffs

With the wire framing in place (Section **??**), Echo apps still need a clear contract for how to *interpret* WARP-related messages. This section specifies how engines, the session host, and client apps exchange WARP snapshots and diffs, and what each participant must do to stay in lockstep.

At a protocol level, this is the same contract used by the Echo Session Service and the WARP Viewer: Section **??** explains how the host enforces gapless streams and fans out frames, while Section **??** shows one concrete tool (the WARP Viewer) that consumes these streams and projects them into a 3D ring of WARPs.

8.4.1 Roles and Responsibilities

Engine Owns the authoritative WARP state and emits structural changes per epoch.

Session Host Acts as a hub and history keeper:

- Tracks one stream per `WarpId` (worldline).
- Enforces gapless epoch sequences per `WarpId`.
- Fans out WARP frames to subscribed clients.

Client App Any tool or runtime that wants to observe or publish an WARP: game, viewer, inspector, profiler, ...

8.4.2 Role Summary

Table **??** summarizes the responsibilities of each role in the WARP streaming pipeline.

Role	Responsibilities
Engine	Owns the authoritative WARP; emits epoch-aligned snapshots and diffs. Computes canonical state hashes after each commit. Guarantees that each `Diff` maps from epoch k to $k+1$.
Session Host	Tracks per-`WarpId` streams (`last_epoch`, `last_hash`, latest snapshot, subscribers, producer). Enforces gapless diffs (`from = last_epoch`, `to = from + 1`). Fans out accepted frames to all subscribers for that `WarpId`.
Client App	Maintains local per-`WarpId` state (epoch, graph, history). Applies the snapshot/diff algorithm in this section. Treats any gap or hash mismatch as a desync and drops the stream. Projects the resulting WARPs into its own UI/logic (e.g. a 3D ring, inspector view, debug overlay).

Table 8.2: Summary of WARP streaming roles and their responsibilities.

8.4.3 WARP Frames

At the JS-ABI level, WARP traffic is carried as:

- `op = "warp_stream"` payloads containing $\langle$`warp_id`, `frame`$\rangle$.
- `frame` is either:
  - **Snapshot**: {`epoch`, `graph: RenderGraph`, `state_hash?`}.
  - **Diff**: {`from_epoch`, `to_epoch`, `ops: Vec<WarpOp>`, `state_hash?`}.
- `WarpOp` is a structural graph mutation: Add/Update/Remove node or edge with a data patch.

Each `WarpId` corresponds to a single logical stream with at most one producer and many consumers.

8.4.4 Consumer State Machine

Applications that consume WARP streams follow a small state machine per `WarpId`. Figure **??** shows the idealized flow.

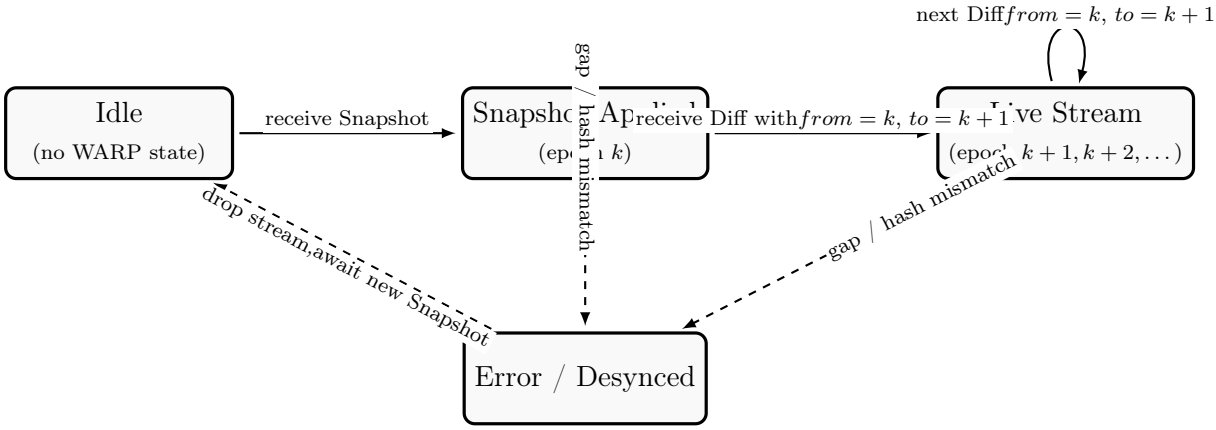

Figure 8.3: Per-WARP consumer state machine for snapshots and diffs.

8.4.5 Per-WARP Consumer Algorithm

For each subscribed `WarpId`, a client app maintains:

- The most recent epoch number e (or *None* before a snapshot).
- A local `RenderGraph` mirror of the stream.
- Optional cached `state_hash`.

On frames for that `WarpId`:

Snapshot frame

1. Replace the local `RenderGraph` with the snapshot graph.
2. Set $e =$ `snapshot.epoch`.
3. If a `state_hash` is present, recompute a canonical hash of the local graph and compare. Mismatch $\Rightarrow$ enter "Error" state.
4. Rebuild any renderable scene or indices needed by the app.

Diff frame

1. If e is *None*, treat this as a protocol error: clients must see a snapshot first.
2. Enforce gapless epochs:
   - Require `diff.from_epoch` $= e$.
   - Require `diff.to_epoch` $= e + 1$.

Any violation ⇒ enter "Error" state and drop the stream.

3. Apply each `WarpOp` in order to the local `RenderGraph`.
4. Set $e =$ `diff.to_epoch`.
5. If `state_hash` is present, recompute and compare. Mismatch ⇒ enter "Error" state.
6. Rebuild or incrementally update the app's renderable scene.

8.4.6 Engine and Session Host Responsibilities

The engine and session host together ensure that clients can safely apply the algorithm above:

- **Engine**:
  - Emits structural diffs (`WarpOp`) that map exactly from epoch k to $k + 1$.
  - Computes a canonical `state_hash` after each commit.
- **Session Host**:
  - Tracks `last_epoch` and `last_hash` per `WarpId`.
  - On producer input:
    - Accepts a **Snapshot** as a reset for that stream.
    - Accepts a **Diff** only if: `from_epoch` = `last_epoch` and `to_epoch` = `from_epoch` + 1.
  - Fan-outs each accepted frame to all subscribers of that `WarpId`.

Clients can therefore assume:

- Each subscribed stream is gapless and totally ordered.
- Each WARP frame is self-identified by its `WarpId`.
- A fresh snapshot always provides a safe starting point after disconnect or error.

8.4.7 Multi-WARP Applications

An application may subscribe to multiple `WarpId`s at once. The recommended pattern is:

- Maintain a map `WarpId` ↦ `WarpRenderState`.
- For each incoming frame, route it to the appropriate entry and run the per-WARP consumer algorithm.
- Keep a separate "view model" (e.g., the WARP ring used by the viewer) that decides how to arrange and highlight the various WARPs in the UI.

This keeps the network-facing logic uniform across apps: everyone reuses the same snapshot/diff semantics, while higher-level tools can choose their own visualization and interaction patterns.

Aside: WARP Viewer as a Consumer The Echo WARP Viewer (Section **??**) is a direct application of this pattern:

- It maintains per-`WarpId` render state (graph, epoch, history).
- It routes frames from the session service into the per-WARP algorithm above, treating any epoch gap or hash mismatch as a fatal desync.
- Its UI layer simply arranges those WARPs on a ring and lets the user pick which one is in focus, without changing the underlying network contract.

Part III

Math and Proofs

Chapter 9

Deterministic Mathematics

The cornerstone of Echo's reliability is its deterministic math module. Standard IEEE 754 floating-point arithmetic is famously non-deterministic across different architectures and compiler optimizations.

9.1 The Hazards of Hardware Floats

- **NaN Payloads:** The bit pattern of 'NaN' (Not a Number) is not fully specified. '0.0 / 0.0' might produce different bits on x86 vs ARM.
- **Signed Zero:** '-0.0' and '+0.0' compare equal but hash differently.
- **FMA (Fused Multiply-Add):** Some CPUs combine multiply and add into one step, changing rounding results.
- **Transcendental Functions:** 'sin()', 'cos()', 'pow()' are often implemented in libc or hardware microcode, with varying precision.

9.2 Echo's Current Float Wrapper: `F32Scalar`

Echo wraps 'f32' in a strict type, 'F32Scalar', with the policies implemented today:

1. **Canonical Zero:** The constructor maps '-0.0' to '+0.0' (see Listing **??**).
2. **NaN passthrough (for now):** NaNs are preserved as-is; canonical NaN collapsing is a planned follow-up.
3. **Transcendentals:** 'sin'/'cos' currently call 'f32::sin'/'f32::cos'. Deterministic LUT or polynomial replacements are planned to remove platform variance.

Listing 9.1: F32Scalar canonicalizes -0.0 and derives ordering

```rust
#[derive(Debug, Copy, Clone)]
pub struct F32Scalar {
    value: f32, // invariant: never stores -0.0
}

impl F32Scalar {
```

```
    pub const fn new(num: f32) -> Self {
        Self { value: num + 0.0 } // -0.0 -> +0.0
    }
}

impl Ord for F32Scalar {
    fn cmp(&self, other: &Self) -> Ordering {
        self.value.total_cmp(&other.value)
    }
}
```

9.3 Geometric Types

Built on top of ‘F32Scalar‘ are the standard geometric types:

- `Vec2`, `Vec3`, `Vec4`
- `Mat3`, `Mat4` (Column-major)
- `Quat` (Quaternions for rotation)

These types ensure that vector operations (dot product, cross product, matrix multiplication) propagate determinism.

9.4 Fixed Point Option

For environments where floating-point hardware is completely unreliable or missing, Echo supports a ‘DFix64‘ (Deterministic Fixed Point) backend. This uses ‘i64‘ to represent decimal numbers (e.g., 32 bits for integer, 32 bits for fraction).

Part IV

Ops and Governance

Chapter 10

Ops and Governance (Starter)

Echo favors merge-only history, deterministic builds, and documented intent.

10.1 Working Agreements

- No rebases or force pushes; use the branch merge playbook.
- Update relevant ADRs and specs when runtime behavior changes.
- Run `cargo clippy -all-targets` and `cargo test` before PRs; Docs Guard expects updated metadata.

10.2 CI / Docs Guard

- Hooks run fmt, clippy, docs guard, and SPDX checks.
- GitHub Actions enforce SPDX headers and LaTeX docs builds.

10.3 Next Additions

- Determinism incident runbook.
- Branch merge decision tree (with TikZ diagram).
- Telemetry/diagnostics quickstart.

Part V

Echo Editor Tools

Chapter 11

Echo Editor Constellation of Services

Echo's tooling isn't a single monolithic editor. Instead, we run a constellation of small, purpose-built tools around a shared *Editor Service Hub*. Each tool speaks three channels: Record Model Graph (WARP) state (diffs/snapshots), notifications/toasts, and commands/edits. As shown in Figure **??**, the hub maintains session authority and causal ordering so every participant sees a consistent, deterministic timeline.

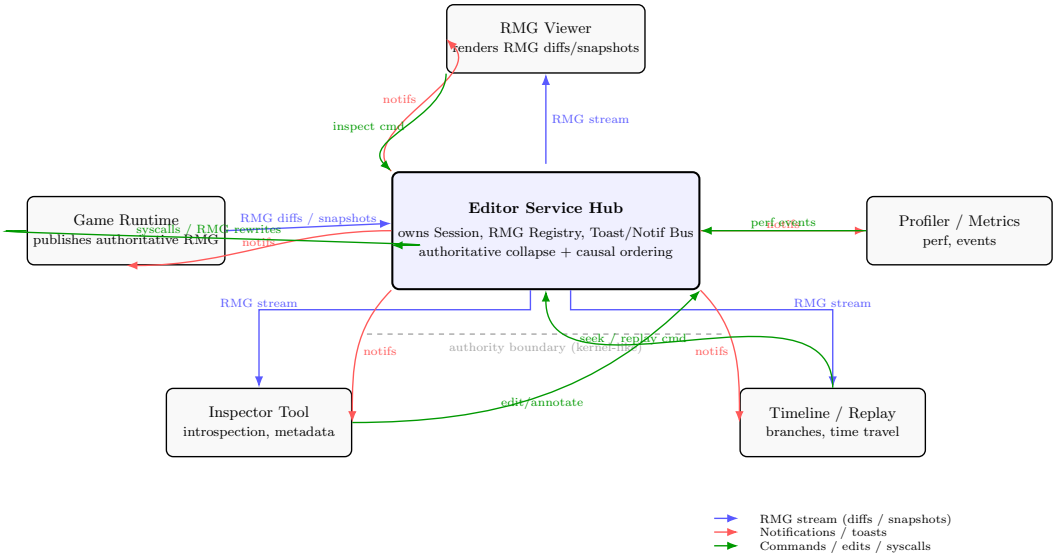

Figure 11.1: Editor constellation: the hub owns sessions and WARP registry; tools subscribe to WARP streams, receive notifications, and send commands.

Roles

- **Editor Service Hub**: authoritative session, WARP registry, notification bus; enforces causal order and collapse rules.
- **Publishers** (e.g., Game Runtime): stream WARP diffs/snapshots into the hub.
- **Consumers** (Viewer, Inspector, Timeline, Profiler): subscribe to chosen WARP streams; render, inspect, or replay; emit commands back.

Interfaces (hex-friendly)

- WARP Stream Port: ordered diffs + on-demand snapshots per WARP id.
- Notification Port: broadcast info/warn/error events (toasts/logs) scoped to session or WARP.
- Command Port: edits, inspections, timeline seeks, profiler signals; acknowledged with deterministic acks.

Why this shape It keeps UI and runtime concerns decoupled, supports multi-tool workflows (viewer + inspector + profiler simultaneously), and preserves determinism via a single authority boundary. Every tool remains a hexagonal adapter; the hub is just another adapter hosting the shared core.

11.1 Echo Session Service and WARP Viewer Sync

Echo's editor constellation streams WARP state from the engine to tools over a gapless, deterministic protocol. This section documents the session service, the wire format, and how the WARP Viewer consumes snapshots/diffs to stay in lockstep.

For a role-level summary of how the Engine, Session Host, and client apps share responsibility for WARP streaming, see the generic consumer contract in Section **??** and Table **??**.

11.1.1 Roles

- **Engine / Session Service**: authoritative owner of the WARP; emits snapshots and diffs over a Unix socket; guarantees ordered, gapless epochs.
- **WARP Viewer**: connects to the service, receives frames, applies structural ops, renders the graph; treats any epoch gap/hash mismatch as a protocol error.
- **Other tools**: inspectors, profilers, etc., use the same frames and can render or analyze the graph identically.

11.1.2 Wire Types (from `echo-graph` / `echo-session-proto`)

- **EpochId**: monotonically increasing tick id.
- **WarpFrame**: either `Snapshot` or `Diff`.
- **Snapshot**: `{ epoch, graph: RenderGraph, state_hash? }` (first frame after connect).
- **Diff**: `{ from_epoch, to_epoch, ops: Vec<WarpOp>, state_hash? }` with `to = from + 1` in live streams.
- **WarpOp**: structural graph mutations (Add/Update/Remove node/edge with payload patches).
- **Notification**: info/warn/error scoped to Global/Session/WARP/Local; carried alongside WarpFrame.

11.1.3 Protocol Invariants

- First frame must be a Snapshot.
- Live stream is gapless: `diff.from_epoch == local_epoch`, `diff.to_epoch == local_epoch + 1`. Any violation = protocol error + disconnect.
- Optional state hashes are computed over canonical RenderGraph (sorted, CBOR) with blake3; mismatch = error.
- Late join/reconnect: service sends a fresh Snapshot (or Snapshot + diffs if it can resume); viewer resets to the Snapshot epoch.

11.1.4 Viewer Apply Loop (gapless)

```
Snapshot(s):
  wire_graph = s.graph
  epoch = s.epoch
  scene = scene_from_wire(wire_graph)
  if hash provided -> verify
  screen = View

Diff(d):
  assert d.from == epoch
  assert d.to   == epoch + 1
  for op in d.ops: wire_graph.apply_op(op)
  epoch = d.to
  if hash provided -> verify
  scene = scene_from_wire(wire_graph)
```

11.1.5 Service Emit Loop

1. On client connect: send **Snapshot** of current epoch, then begin diffs.
2. On each tick: compute structural ops from epoch k to $k + 1$, emit **Diff** with `state_hash` of post-state.
3. Ordering is the transport contract: one socket, ordered CBOR frames, length-prefixed.

11.1.6 Failure Handling

- Any gap or hash mismatch: surface an error toast/banner; drop the connection; prompt reconnect.
- Socket read/parse failure: drop connection; return to Title screen.

11.1.7 Why Structural Ops (not rule replay)

- Viewer stays lightweight and version-tolerant; no need to ship rule code.
- Determinism is preserved because ops are applied in recorded order with hashes to verify.

11.1.8 Next Steps

- Engine emitter: send real RenderGraph snapshots/diffs using `echo-graph` types.
- Viewer: improve `scene_from_wire` to honor payload-driven layout/appearance.
- Add resume-from-epoch handshake (`WarpHello`) to support reconnect without full snapshot when logs are retained.

11.2 Echo WARP Viewer: State Machine & UI

The WARP Viewer is a thin, rendering-first client. It connects to the Echo Session Host, subscribes to one or more WARP streams, and presents a minimal HUD over a 3D scene. This section fixes the viewer's states, transitions, and on-screen layout so the implementation and the design stay in lockstep.

At the protocol level, the Viewer is "just" a client app in the WARP streaming pipeline. It follows the generic snapshot/diff consumer contract in Section **??** and sits in the "Client App" row of Table **??**; what is specific here is *how* it projects those WARPs into a 3D ring and UI.

11.2.1 Overview

- Full-screen startup flow (Title → Connecting → Viewer).
- Optional Settings overlay reused between Title and Viewer.
- Viewer HUD: Menu button, toasts stack, perf/controls/stats blocks, persistent watermark/version.
- Multiple WARPs shown on a ring around the camera; arrow keys rotate the ring to bring one WARP to the front.
- Menu overlay offers: Settings, Publish Local WARP, Subscribe to WARP, Back.

11.2.2 Top-Level States

Start Ephemeral entry; immediately transitions to Title.

Title Wordmark + version + primary menu (Connect / Settings / Exit).

Connecting Full-screen status with boot-like log lines while the session client handshakes.

Viewer 3D scene + HUD. Overlays (Menu, Settings, WARP Directory) sit atop this state.

Error Optional full-screen error (e.g., desync) with a path back to Title.

11.2.3 Transitions

- Start → Title on launch.
- Title → Connecting when Connect is pressed.
- Title → Title after Settings (Save/Back) or if connection fails/cancelled.
- Title → Exit when Exit is pressed.
- Connecting → Viewer on successful hello + directory fetch.
- Connecting → Title on failure or cancel.
- Viewer → Title on explicit disconnect.
- Any → Error on fatal protocol/stream violation; Error → Title via Back.

11.2.4 State Machine Diagram

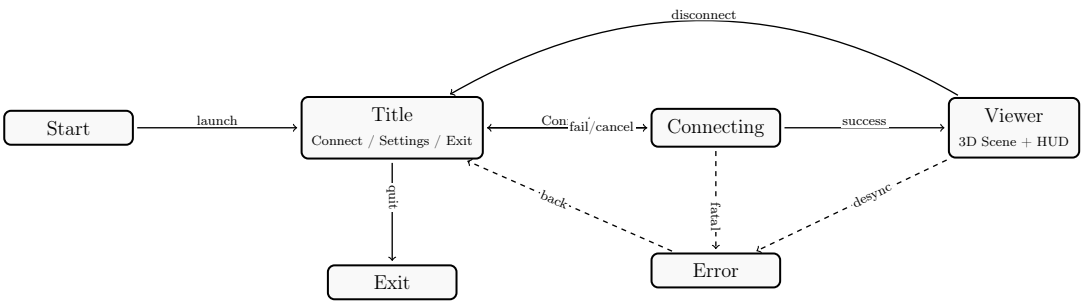

Figure 11.2: Top-level WARP Viewer state machine.

11.2.5 Screen Layouts (Wireframes)

All layouts are schematic; exact styling is left to implementation.

Title Full-screen with wordmark/version and vertical menu (Connect, Settings, Exit). Selecting Connect shows host/port fields + Connect button in-place; Settings shows app settings with Save/Back; Exit quits.

Connecting Cleared screen with wordmark/version bottom-right and a vertically stacked log, e.g.:

- Connecting...
- Connected. Sending hello...
- Session started.

- Requesting WARP directory...
- WARP indexes received.

Transitions to Viewer on success.

Viewer Full-screen 3D scene with HUD anchors:

- **Menu** button (top-left) opens overlay: Settings / Publish Local WARP / Subscribe to WARP / Back.
- **Toasts** stack (top-right).
- **Perf** block (bottom-left) showing FPS/timings (toggled in Settings).
- **Controls** block (bottom-left) showing WASD/QE, mouse look, right-drag spin, wheel zoom, arrow-keys cycle WARPs.
- **Stats** block (bottom-center/right) showing WARP id/epoch, subscription status.
- **Watermark/version** always bottom-right.

Overlay logic: Menu, Settings, and WARP Directory sit atop Viewer; closing them restores the HUD.

11.2.6 WARP Ring Layout

WARPs (local + subscribed) are placed on a horizontal ring of radius R around the camera. For n WARPs, positions are

$$x_i = R\cos\theta_i, \quad z_i = R\sin\theta_i, \quad \theta_i = \frac{2\pi i}{n} + \theta_{\text{offset}}.$$

Arrow keys adjust θ_{offset} to rotate the ring so a chosen WARP sits in front of the camera. Adding subscriptions increases n and redistributes positions evenly.

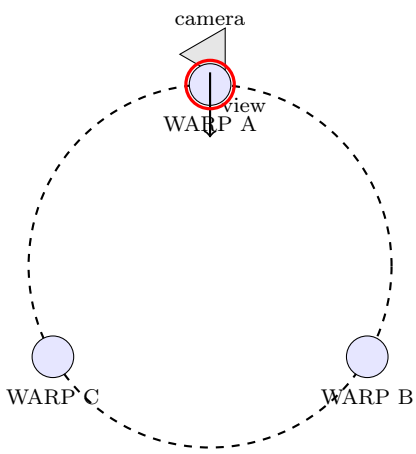

Figure 11.3: Overhead ring layout; arrow keys rotate which WARP is active.

11.2.7 Publish / Subscribe semantics

- **Publish Local WARP**: register as the sole producer for a chosen WarpId and stream Snapshot → gapless Diff to the host.
- **Subscribe to WARP**: open the WARP Directory overlay (listing host-known WarpIds); selecting one sends SubscribeWarp and adds that WARP to the ring; arrow keys cycle which WARP is in front.

11.2.8 Notes

- Error handling: on desync/hash mismatch, drop to Error or Title with a clear reconnect path; keep the watermark visible.
- All overlays reuse the same settings form (host/port, warp id, HUD toggles) to avoid divergent UIs.

11.3 Echo Tool Hexagon Pattern

Echo tools (viewer, inspector, profiler, . . .) should all follow the same hexagonal pattern:

- Domain logic and state live in a pure core (no direct IO).
- IO happens only through narrow ports (config, session, render).
- Each port has one or more shared adapters that talk to concrete libraries (filesystem, sockets, wgpu, egui, . . .).
- `main.rs` + the top-level `App` act as thin glue: wiring ports and reducers, delegating work to the core.

The WARP Viewer is a concrete example of this architecture, and can serve as a template for future tools.

11.3.1 Ports and Adapters

At a high level, most Echo tools need three capabilities:

ConfigPort Load/save structured settings and user preferences.

SessionPort Receive WARP snapshots/diffs and notifications from the session hub.

RenderPort Request redraws from whatever windowing/surface system is in use.

Figure **??** shows these ports and their adapters for the WARP Viewer.

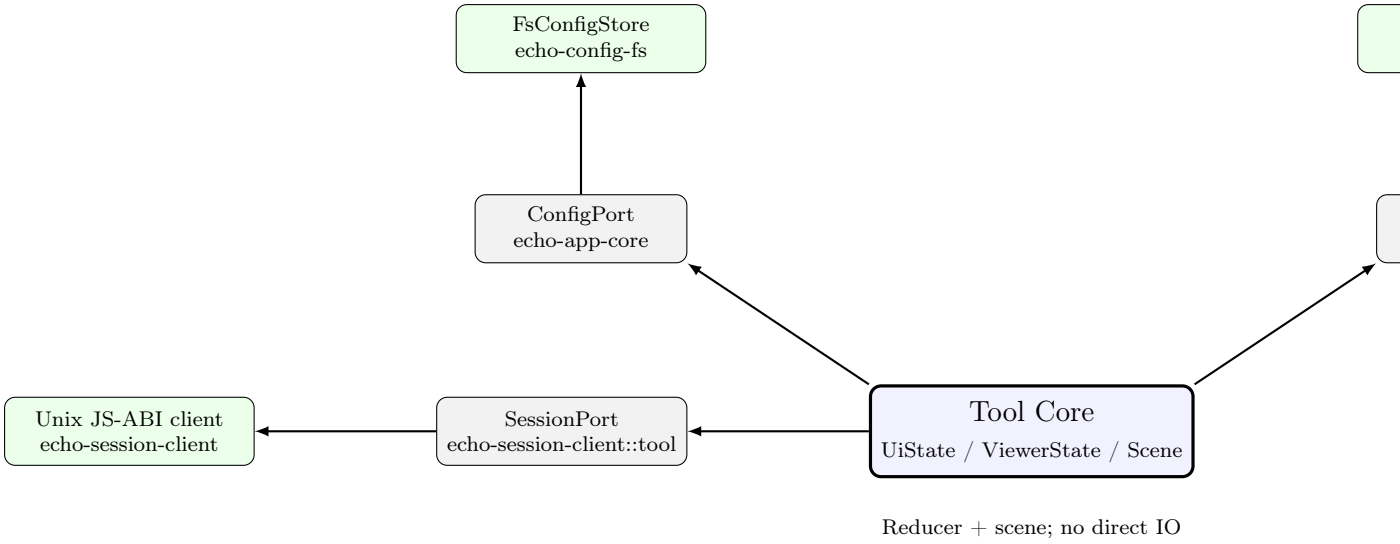

Figure 11.4: Tool hexagon pattern: core + ports + adapters (WARP Viewer example).

11.3.2 WARP Viewer as Reference Implementation

For the WARP Viewer specifically (Section **??**):

- **Core**:
  - `UiState` and `UiEvent`/`UiEffect` in `ui_state.rs`.
  - `ViewerState` and WARP scene representation in `viewer_state.rs`/`scene.rs`.
  - Session frame handling (snapshots/diffs) in `session_logic.rs` (see also Section **??**).
- **ConfigPort**:
  - Provided by `echo-app-core::ConfigPort`.
  - Implemented by `echo-config-fs::FsConfigStore` + `ConfigService`, wired into `App` in `app.rs`.
- **SessionPort**:
  - Defined in `echo-session-client::tool` as `SessionPort`.
  - Implemented by `ChannelSession`, which wraps the `WarpFrame` / `Notification` channels returned by `connect_channels_for`.
  - The viewer's `UiEffectsRunner` uses this to connect and report errors (via `UiEvent::ShowError`), and the frame loop drains it into `session_logic::process_frames`.
- **RenderPort**:
  - Defined in `echo-app-core::render_port::RenderPort`.
  - Implemented by a small `WinitRenderPort` wrapper around `winit::window::Window::reques`
  - The viewer calls `RenderPort::request_redraw` after each frame, leaving windowing details outside the core.

- **UI Rendering**:
  - Stateless egui helpers in `ui.rs` draw Title/Connecting screens and the Viewer HUD, driven entirely by `UiState` and `ViewerState`.
  - All input flows through the reducer (`UiEvent` → `UiState` + `UiEffect`); effects are turned into port calls by `UiEffectsRunner`.

11.3.3 Guidelines for Future Tools

When building new tools on the Echo stack:

- Define a domain-specific `UiState` and `UiEvent`/`UiEffect` pair.
- Keep the reducer pure: only change state and emit effects; do not perform IO inside the reducer.
- Route all side effects (config, session, rendering) through ports: `ConfigPort`, `SessionPort`, `RenderPort`, or additional ports defined in shared crates.
- Implement adapters in small, dedicated modules or crates (e.g., a filesystem store, a Unix session client, a WGPU renderer), and keep `main.rs` + `App` as thin glue.
- Use the WARP Viewer layout in this part of the book as a worked example, and the JS-ABI/WARP streaming specs (Sections **??** and **??**) as the canonical contract for session traffic.

11.3.4 Crate Map for Tools

Table **??** summarizes the primary tool-related crates and their responsibilities. This map is intentionally small; future crates should aim to fit naturally alongside these roles.

Crate	What it does
`echo-app-core`	Shared app services and ports for tools: configuration service (`ConfigService`/`ConfigPort`), toast notifications (`ToastService`), and the `RenderPort` trait for redraw requests. Keeps UI/runtime adapters thin and framework-agnostic.
`echo-config-fs`	Filesystem-backed implementation of `ConfigStorePort` using platform config directories and JSON. Used by tools to persist preferences (camera pose, HUD toggles, host settings).
`echo-session-proto`	Logical wire schema for the Echo session hub: `Message`, `OpEnvelope`, deterministic JS-ABI framing helpers, and `Notification`/`WarpStream` types that wrap `echo-graph` WARP frames.
`echo-session-service`	Headless session hub process: listens on a Unix socket, handles handshakes, tracks per-`WarpId` stream state, enforces gapless snapshots/diffs, and fans out frames/notifications to subscribers.
`echo-session-client`	Unix-socket JS-ABI client: connects to the hub, sends handshakes and subscriptions, and decodes packets into `Message` values. Includes the `tool` module with channel-based `SessionPort` + `ChannelSession` for tools.
`warp-viewer`	3D WARP viewer built on the above ports and adapters: uses `echo-session-client::tool` for session traffic, `echo-app-core` + `echo-config-fs` for prefs, and a WGPU renderer for visualization. Acts as the reference implementation of the tool hexagon pattern.

Table 11.1: Tool-related crates and their roles in the Echo stack.

Part VI

Tour de Code

Chapter 12

Tour de Code: Overview

The complete function-by-function trace of Echo's execution pipeline.

This tour traces every function call involved in processing a user action through the Echo engine. File paths and line numbers are accurate as of 2026-01-18.

Welcome to the Tour de Code. I'm going to walk you through this codebase like we're pair programming. When I see something clever, I'll tell you why it's clever. When there's a non-obvious design decision, I'll explain the trade-offs.

The goal: by the end of this tour, you'll understand not just *what* the code does, but *why* it's structured the way it is.

12.1 The Pipeline at a Glance

A user action flows through Echo in eight stages:

1. **Intent Ingestion** — User action becomes an `Intent`
2. **Transaction Lifecycle** — Intents are batched into transactions
3. **Rule Matching** — Rules produce `PendingRewrite`s
4. **Scheduler: Drain & Reserve** — Radix sort + independence check
5. **BOAW Parallel Execution** — Workers execute in parallel
6. **Delta Merge** — Thread-local deltas merge canonically
7. **Hash Computation** — State root, patch digest, commit hash
8. **Commit Orchestration** — Final state materialization

The Big Picture: Every stage is designed around one principle: **determinism**. Given the same inputs, the engine must produce identical outputs regardless of thread scheduling, CPU count, or platform.

> This isn't just about correctness—it enables replay, forking, and distributed consensus.

12.2 Key Files

Component	Primary File
Engine core	`crates/echo-core/src/engine.rs`
Transaction handling	`crates/echo-core/src/transaction.rs`
Rule matching	`crates/warp-core/src/rule_matching.rs`
Scheduler (Radix)	`crates/warp-core/src/radix_scheduler.rs`
BOAW execution	`crates/warp-core/src/boaw/exec.rs`
Shard routing	`crates/warp-core/src/boaw/shard.rs`
Delta/merge	`crates/warp-core/src/tick_delta.rs`
Hashing	`crates/warp-core/src/hashing.rs`

12.3 Reading This Tour

Each section follows a consistent format:

- **Entry Point** — The function that kicks off this stage
- **Call Trace** — Indented tree showing the execution flow
- **Data Structures** — Key types involved
- **Commentary** — Why it works this way

> **Pro Tip:** Keep a terminal open with the codebase. When you see a file reference like `src/boaw/exec.rs:61-83`, jump to that location and read the actual code alongside this tour.

Chapter 13

Intent Ingestion

Entry Point: `Engine::ingest_intent()`
File: `crates/warp-core/src/engine_impl.rs:1216`

> **Tour Guide:** This is where user actions enter the system. Notice how Echo treats intents as *immutable, content-addressed* data from the very first moment. The intent bytes are hashed to create a unique identifier, ensuring that duplicate intents are detected automatically—no coordination required.

13.1 Function Signature

```rust
pub fn ingest_intent(&mut self, intent_bytes: &[u8]) -> Result<IngestDisposition,
    EngineError>
```

Returns:

- `IngestDisposition::Accepted { intent_id: Hash }` — New intent accepted
- `IngestDisposition::Duplicate { intent_id: Hash }` — Already ingested

13.2 Complete Call Trace

```
Engine::ingest_intent(intent_bytes: &[u8])
|
+--[1] compute_intent_id(intent_bytes) -> Hash
|      FILE: crates/warp-core/src/inbox.rs:205
|      CODE:
|        let mut hasher = blake3::Hasher::new();
|        hasher.update(b"intent:");           // Domain separation
|        hasher.update(intent_bytes);
|        hasher.finalize().into()             // -> [u8; 32]
|
+--[2] NodeId(intent_id)
|      Creates strongly-typed NodeId from Hash
|
+--[3] self.state.store_mut(&warp_id) -> Option<&mut GraphStore>
|      FILE: crates/warp-core/src/engine_impl.rs:1221
|      ERROR: EngineError::UnknownWarp if None
```

```
|
+--[4] Extract root_node_id from self.current_root.local_id
|
+--[5] STRUCTURAL NODE CREATION (Idempotent)
|     +-- make_node_id("sim") -> NodeId
|     |   FILE: crates/warp-core/src/ident.rs:93
|     |   CODE: blake3("node:" || "sim")
|     |
|     +-- make_node_id("sim/inbox") -> NodeId
|     |   CODE: blake3("node:" || "sim/inbox")
|     |
|     +-- make_type_id("sim") -> TypeId
|     |   FILE: crates/warp-core/src/ident.rs:85
|     |   CODE: blake3("type:" || "sim")
|     |
|     +-- make_type_id("sim/inbox") -> TypeId
|     +-- make_type_id("sim/inbox/event") -> TypeId
|     |
|     +-- store.insert_node(sim_id, NodeRecord { ty: sim_ty })
|     |   FILE: crates/warp-core/src/graph.rs:175
|     |   CODE: self.nodes.insert(id, record)
|     |
|     +-- store.insert_node(inbox_id, NodeRecord { ty: inbox_ty })
|
+--[6] STRUCTURAL EDGE CREATION
|     +-- make_edge_id("edge:root/sim") -> EdgeId
|     |   FILE: crates/warp-core/src/ident.rs:109
|     |   CODE: blake3("edge:" || "edge:root/sim")
|     |
|     +-- store.insert_edge(root_id, EdgeRecord { ... })
|     |   FILE: crates/warp-core/src/graph.rs:188
|     |   +-- GraphStore::upsert_edge_record(from, edge)
|     |       FILE: crates/warp-core/src/graph.rs:196
|     |       UPDATES:
|     |         self.edge_index.insert(edge_id, from)
|     |         self.edge_to_index.insert(edge_id, to)
|     |         self.edges_from.entry(from).or_default().push(edge)
|     |         self.edges_to.entry(to).or_default().push(edge_id)
|     |
|     +-- store.insert_edge(sim_id, EdgeRecord { ... }) [sim -> inbox]
|
+--[7] DUPLICATE DETECTION
|     store.node(&event_id) -> Option<&NodeRecord>
|     FILE: crates/warp-core/src/graph.rs:87
|     CODE: self.nodes.get(id)
|     IF Some(_): return Ok(IngestDisposition::Duplicate { intent_id })
|
+--[8] EVENT NODE CREATION
|     store.insert_node(event_id, NodeRecord { ty: event_ty })
|     NOTE: event_id = intent_id (content-addressed)
|
+--[9] INTENT ATTACHMENT
|     +-- AtomPayload::new(type_id, bytes)
|     |   FILE: crates/warp-core/src/attachment.rs:149
|     |   CODE: Self { type_id, bytes: Bytes::copy_from_slice(intent_bytes) }
|     |
|     +-- store.set_node_attachment(event_id, Some(AttachmentValue::Atom(payload)))
|         FILE: crates/warp-core/src/graph.rs:125
```

```
|         CODE: self.node_attachments.insert(id, v)
|
+--[10] PENDING EDGE CREATION (Queue Membership)
|      +-- pending_edge_id(&inbox_id, &intent_id) -> EdgeId
|      |   FILE: crates/warp-core/src/inbox.rs:212
|      |   CODE: blake3("edge:" || "sim/inbox/pending:" || inbox_id || intent_id)
|      |
|      +-- store.insert_edge(inbox_id, EdgeRecord {
|             id: pending_edge_id,
|             from: inbox_id,
|             to: event_id,
|             ty: make_type_id("edge:pending")
|         })
|
+--[11] return Ok(IngestDisposition::Accepted { intent_id })
```

Clever Pattern: Domain Separation in Hashing

Notice step [1]: the hasher prefixes with `b"intent:"` before the actual data. This is a cryptographic best practice called *domain separation*—it prevents a hash collision between an intent and, say, a node ID that happens to have the same bytes.

Echo uses this pattern consistently:

- `"intent:"` for intent IDs
- `"node:"` for node IDs
- `"type:"` for type IDs
- `"edge:"` for edge IDs

This ensures that even if two different domain values have the same raw bytes, they'll produce different hashes.

Deep Dive: Why Content-Addressed Event IDs?

In step [8], note that `event_id = intent_id`. This is a profound design choice:

1. **Automatic deduplication**: If the same intent arrives twice, it hashes to the same ID, and step [7] catches it.
2. **Reproducibility**: Given the same intent bytes, any node in a distributed system will compute the same event ID.
3. **Auditability**: You can verify an event's integrity by re-hashing its content.

This is the foundation of Echo's deterministic execution model—events are identified by *what they are*, not *when they arrived*.

13.3 Data Structures Modified

Structure	Field	Change
`GraphStore`	`nodes`	+3 entries (sim, inbox, event)
`GraphStore`	`edges_from`	+3 edges (root→sim, sim→inbox, inbox→event)
`GraphStore`	`edges_to`	+3 reverse entries
`GraphStore`	`edge_index`	+3 edge→from mappings
`GraphStore`	`edge_to_index`	+3 edge→to mappings
`GraphStore`	`node_attachments`	+1 (event → intent payload)

Tour Guide: Notice the **four separate edge indices**: `edges_from`, `edges_to`, `edge_index`, and `edge_to_index`. This redundancy enables O(1) lookups in any direction—find edges from a node, to a node, or look up either endpoint given an edge ID. The space cost is modest (pointers/IDs are small), but the query flexibility is enormous.

Chapter 14

Transaction Lifecycle

14.1 Begin Transaction

Entry Point: `Engine::begin()`
File: `crates/warp-core/src/engine_impl.rs:711-719`

```rust
pub fn begin(&mut self) -> TxId {
    self.tx_counter = self.tx_counter.wrapping_add(1);
    if self.tx_counter == 0 {
        self.tx_counter = 1;  // Zero is reserved
    }
    self.live_txs.insert(self.tx_counter);
    TxId::from_raw(self.tx_counter)
}
```

> **Watch Out: The Zero Invariant**: `TxId(0)` is reserved as an invalid/sentinel value. Without line 715's check, after 2^{64} transactions the counter would wrap to zero and confuse code that uses zero to mean "no transaction." Defensive programming at its finest.

```
Engine::begin()
|
+-- self.tx_counter.wrapping_add(1)
|   Handles u64::MAX → 0 overflow
|
+-- if self.tx_counter == 0: self.tx_counter = 1
|   INVARIANT: TxId(0) is reserved as invalid
|
+-- self.live_txs.insert(self.tx_counter)
|   TYPE: HashSet<u64>
|
+-- TxId::from_raw(self.tx_counter)
    TYPE: #[repr(transparent)] struct TxId(u64)
```

> The `#[repr(transparent)]` on `TxId` guarantees the same memory layout as `u64`. You get type safety (can't accidentally pass a `NodeId` where a `TxId` is expected) with no runtime overhead.

14.2 Abort Transaction

Entry Point: `Engine::abort()`
File: `crates/warp-core/src/engine_impl.rs:962-968`

```rust
pub fn abort(&mut self, tx: TxId) {
    self.live_txs.remove(&tx.value());
    self.scheduler.finalize_tx(tx);
    self.bus.clear();
    self.last_materialization.clear();
    self.last_materialization_errors.clear();
}
```

> Abort is refreshingly simple—just remove the transaction from tracking and clear transient state. No rollback needed because Echo hasn't mutated the graph yet! All graph mutations happen atomically during commit. This is a key architectural decision: the graph is effectively immutable until commit time.

Chapter 15

Rule Matching

Entry Point: `Engine::apply()`
File: `crates/warp-core/src/engine_impl.rs:730-737`

> **The Big Picture:** Now we enter the heart of Echo's reactive model. Rules are matched against graph patterns, and when they match, they're enqueued for execution. The beauty is that matching is *pure*—it reads the graph but doesn't modify it.

15.1 Function Signature

```rust
pub fn apply(
    &mut self,
    tx: TxId,
    rule_name: &str,
    scope: &NodeId,
) -> Result<ApplyResult, EngineError>
```

15.2 Call Trace

```
Engine::apply(tx, rule_name, scope)
|
+-- self.rules.get(rule_name) → Option<&Rule>
|   FILE: crates/warp-core/src/rules.rs
|   Returns None if rule not registered
|
+-- rule.match_at(view, scope) → Vec<Match>
|   FILE: crates/warp-core/src/rule_matching.rs:45-89
|   |
|   +-- FOR pattern IN rule.patterns:
|       |
|       +-- pattern.try_match(view, scope) → Option<Match>
|       |   Evaluates pattern predicates against graph
|       |
|       +-- IF matched: matches.push(Match { ... })
|
```

```
+-- FOR (idx, m) IN matches.enumerate():
    |
    +-- self.scheduler.enqueue(PendingRewrite {
            tx,
            rule_id: rule.id(),
            scope: *scope,
            match_ix: idx,
            footprint: m.footprint.clone(),
        })
```

15.3 PendingRewrite Structure

File: `crates/warp-core/src/pending.rs:12-24`

```rust
pub struct PendingRewrite {
    pub tx: TxId,
    pub rule_id: RuleId,
    pub scope: NodeId,
    pub match_ix: usize,       // Which match (0..N)
    pub footprint: Footprint,  // Read/write sets
}
```

> The `match_ix` field is subtle but important. A single rule can match multiple times at the same scope (e.g., a "connect all neighbors" rule). Each match becomes a separate `PendingRewrite` with a different `match_ix`.
>
> This index is part of the canonical sort key, ensuring deterministic ordering even when multiple matches occur.

15.4 Footprint Computation

File: `crates/warp-core/src/footprint.rs`

```rust
pub struct Footprint {
    pub n_read: BTreeSet<NodeId>,   // Nodes read
    pub n_write: BTreeSet<NodeId>,  // Nodes written
    pub e_read: BTreeSet<EdgeId>,   // Edges read
    pub e_write: BTreeSet<EdgeId>,  // Edges written
    pub factor_mask: u64,           // Coarse prefilter
}
```

> **The Big Picture:** The footprint is the foundation of BOAW's parallel safety. Two rewrites can execute in parallel if and only if their footprints don't overlap (read-read is allowed, but read-write or write-write conflicts).
>
> The `factor_mask` is a bloom-filter-like optimization: if the masks don't overlap, the footprints definitely don't conflict, allowing fast rejection without checking every element.

Pro Tip: When designing rules, minimize footprint size. A rule that reads the entire graph can never run in parallel with anything. A rule that touches only its immediate neighbors can parallelize heavily.

Chapter 16

Scheduler: Drain & Reserve

> **The Big Picture:** The scheduler is the traffic cop of Echo. It takes a queue of pending rewrites and determines which ones can execute in this tick without conflicting. The output is a deterministic list of `ExecItem`s ready for BOAW.

16.1 Drain Phase (Radix Sort)

Entry Point: `RadixScheduler::drain()`
File: `crates/warp-core/src/radix_scheduler.rs:156-198`

```
RadixScheduler::drain() → Vec<PendingRewrite>
|
+-- items = self.pending.drain(..).collect()
|   Empties the pending queue
|
+-- radix_sort_stable(&mut items)
|   FILE: crates/warp-core/src/radix_scheduler.rs:89-120
|   |
|   +-- Sort key: (scope_hash, compact_rule, nonce)
|       |
|       +-- scope_hash: u64 from NodeId bytes
|       |   CODE: u64::from_le_bytes(scope.as_bytes()[0..8])
|       |
|       +-- compact_rule: u32
|       |   CODE: (rule_id.0 as u32) << 16 | (match_ix as u32)
|       |
|       +-- nonce: u32
|           Transaction-local uniquifier
|
+-- RETURN items (now in canonical order)
```

> Why radix sort instead of quicksort?
>
> - **Determinism**: Radix sort is inherently stable—equal keys maintain their original order. Quicksort's pivot selection can introduce non-determinism.

- **Performance**: For fixed-size keys (we have a 128-bit composite key), radix sort is $O(n)$ instead of $O(n \log n)$.
- **Cache-friendly**: LSD radix sort makes sequential passes through memory.

The sort key packs three values into a single comparable tuple, ensuring that rewrites at the same scope are grouped together, then ordered by rule, then by match index.

16.2 Reserve Phase (Independence Check)

Entry Point: `RadixScheduler::reserve()`
File: `crates/warp-core/src/radix_scheduler.rs:200-256`

```
RadixScheduler::reserve(items: Vec<PendingRewrite>) → Vec<ExecItem>
|
+-- let mut admitted: Vec<ExecItem> = Vec::new()
+-- let mut claimed = GenSet::new()  // Tracks claimed resources
|
+-- FOR item IN items (canonical order):
    |
    +-- IF claimed.conflicts_with(&item.footprint):
    |     // Skip this rewrite (deferred to next tick)
    |     self.pending.push(item);
    |     CONTINUE
    |
    +-- claimed.mark_all(&item.footprint)
    |   Records all read/write targets as claimed
    |
    +-- admitted.push(ExecItem {
           exec: lookup_executor(item.rule_id),
           scope: item.scope,
           origin: OpOrigin::from(&item),
       })
|
RETURN admitted
```

Watch Out: The order matters! Greedy admission in canonical order means the *first* rewrite at each scope wins. If you have two conflicting rewrites, the one with the lower sort key gets admitted, the other is deferred.

This is deterministic, but it can lead to starvation if the same rewrite keeps winning. Echo doesn't currently have fairness guarantees—a frequently-triggered rule can crowd out others.

16.3 GenSet: O(1) Conflict Detection

File: `crates/warp-core/src/genset.rs`

```rust
pub struct GenSet {
```

```
    generation: u64,
    slots: Vec<u64>,  // slot[i] = generation when claimed
}

impl GenSet {
    pub fn mark(&mut self, id: u64) {
        self.slots[id as usize] = self.generation;
    }

    pub fn is_marked(&self, id: u64) -> bool {
        self.slots[id as usize] == self.generation
    }

    pub fn clear(&mut self) {
        self.generation += 1;  // O(1) clear!
    }
}
```

GenSet is a beautiful optimization. Instead of clearing a hash set (which is $O(n)$), we increment a generation counter. An element is "marked" if its slot equals the current generation.

`clear()` becomes $O(1)$. This matters because we call it every tick, and the set can grow large.

The trade-off: we need to pre-allocate slots. For node IDs, we use the lower bits (masked) as the slot index, accepting some false positives in exchange for constant-time operations.

Pro Tip: If you're seeing unexpected conflicts, check your footprint computation. A footprint that's too broad (claiming more than necessary) reduces parallelism. Too narrow, and you'll get data races. The footprint must be a *superset* of actual accesses.

Chapter 17

BOAW Parallel Execution

Entry Point: `execute_parallel()`
File: `crates/warp-core/src/boaw/exec.rs:61-83`

> **The Big Picture:** BOAW—"Best of All Worlds"—is where Echo's determinism meets parallelism. The key insight: **execution order doesn't matter if we sort the outputs**.
>
> Workers execute in arbitrary order on different threads, but their outputs are merged canonically. Same inputs → same outputs, regardless of thread scheduling.

17.1 Entry Point

```rust
pub fn execute_parallel(
    view: GraphView<'_>,
    items: &[ExecItem],
    workers: usize
) -> Vec<TickDelta> {
    assert!(workers >= 1);
    let capped_workers = workers.min(NUM_SHARDS);  // Cap at 256
    execute_parallel_sharded(view, items, capped_workers)
}
```

17.2 Sharding

```
partition_into_shards(items) → Vec<VirtualShard>
|
+-- FOR item IN items:
    |
    +-- shard_of(&item.scope) → usize
    |   CODE:
    |     let bytes = scope.as_bytes();
    |     let first_8 = bytes[0..8];
    |     let val = u64::from_le_bytes(first_8);
    |     (val & 255) as usize  // SHARD_MASK = 255
    |
    +-- shards[shard_id].items.push(item)
```

The sharding is beautifully simple: take the first 8 bytes of the node ID, interpret as a little-endian u64, mask with 255. You get a shard number from 0 to 255.

Why 256 shards?

- **Fine enough**: With random node IDs, work distributes evenly
- **Coarse enough**: Each shard has multiple items, amortizing overhead
- **Power of 2**: Masking is just a bitwise AND, no division needed

Why is this deterministic? Because shard assignment depends only on the node ID, which is content-addressed. The same node always lands in the same shard.

17.3 Work-Stealing Loop

```
std::thread::scope(|s| {
    FOR _ IN 0..workers:
    |
    +-- s.spawn(move || { ... })  // === WORKER THREAD ===
        |
        +-- let mut delta = TickDelta::new()
        |
        +-- LOOP:
            |
            +-- shard_id = next_shard.fetch_add(1, Relaxed)
            |   ATOMIC: Returns old value, increments counter
            |
            +-- IF shard_id >= 256: break
            |
            +-- FOR item IN &shards[shard_id].items:
                +-- (item.exec)(view, &item.scope, &mut delta)
})
```

Each worker runs a loop: atomically claim the next shard number, process all items in that shard, repeat until no shards remain.

See `Ordering::Relaxed`? That's the weakest memory ordering—basically "no synchronization, just do the atomic operation."

Why is that safe here?

1. Each shard is processed by exactly one worker (atomic fetch-add guarantees unique assignment)
2. Workers don't need to see each other's results until after `join()`
3. The `join()` provides the synchronization barrier

Using `Relaxed` instead of `SeqCst` avoids expensive memory barriers. On a 16-core machine, that matters.

Watch Out: The shard claim order is non-deterministic. Worker 1 might claim shard 5 before worker 2 claims shard 3, or vice versa.

This is fine! The merge phase sorts the outputs canonically. The execution order doesn't affect the final result.

But if you're debugging and wondering why execution traces look different between runs, this is why.

17.4 ExecItem Structure

File: `crates/warp-core/src/boaw/exec.rs:19-35`

```rust
#[derive(Clone, Copy)]
pub struct ExecItem {
    pub exec: ExecuteFn,      // fn(GraphView, &NodeId, &mut TickDelta)
    pub scope: NodeId,        // 32-byte node identifier
    pub origin: OpOrigin,     // { intent_id, rule_id, match_ix, op_ix }
}
```

The `ExecItem` is a closure packaged with its context. The `origin` field is crucial for determinism—it's how we sort ops during the merge phase.

Notice `ExecuteFn` is a function pointer, not a `Box<dyn Fn>`. Function pointers are `Copy` and don't allocate. This matters when you're creating thousands of exec items per tick.

17.5 Thread Safety Guarantees

The parallel execution is safe because:

1. **GraphView is read-only**: Workers can't mutate the base state
2. **TickDelta is thread-local**: Each worker has its own delta
3. **Shards are disjoint**: No two workers process the same shard
4. **Merge is sequential**: Happens after all workers join

Pro Tip: If you need to debug parallel execution, set `ECHO_WORKERS=1` to force single-threaded mode. Same results, easier to trace.

Chapter 18

Delta Merge & State Finalization

> **The Big Picture:** After BOAW execution, we have N worker threads each with their own `TickDelta`. The merge phase combines these into a single canonical sequence of operations. This is the "choke point" where parallelism meets determinism.

18.1 Canonical Merge

Entry Point: `merge_deltas()`
File: `crates/warp-core/src/boaw/exec.rs:160-195`

```
merge_deltas(deltas: Vec<TickDelta>) → TickDelta
|
+-- let mut all_ops: Vec<(WarpOp, OpOrigin)> = Vec::new()
|
+-- FOR delta IN deltas:
|   +-- all_ops.extend(delta.ops.into_iter().zip(delta.origins))
|
+-- all_ops.sort_by_key(|(op, origin)| {
|       (op.sort_key(), origin.clone())
|   })
|   |
|   +-- Sort key: (WarpOpKey, OpOrigin)
|       +-- WarpOpKey: (op_kind, target_id)
|       +-- OpOrigin: (intent_id, rule_id, match_ix, op_ix)
|
+-- Dedupe identical ops (same key + same effect)
|
+-- RETURN TickDelta { ops, origins }
```

> The sort key is crucial. `WarpOpKey` groups operations by kind and target (e.g., all `SetAttachment` ops on node X come together). `OpOrigin` breaks ties deterministically—the rewrite that was admitted first (lower intent/rule/match IDs) wins.
>
> Why dedupe? Multiple workers might emit identical operations (e.g., two rules both setting the same attribute to the same value). We keep only one.

> Why sort, not merge? Merge assumes sorted inputs. Our worker deltas are sorted *within* each worker but not *across* workers. A global sort handles both cases.

18.2 WarpOp Sort Key

File: `crates/warp-core/src/warp_op.rs:89-112`

```rust
impl WarpOp {
    pub fn sort_key(&self) -> WarpOpKey {
        match self {
            WarpOp::CreateNode(id) =>
                WarpOpKey::CreateNode(*id),
            WarpOp::DeleteNode(id) =>
                WarpOpKey::DeleteNode(*id),
            WarpOp::SetAttachment { node, key, .. } =>
                WarpOpKey::SetAttachment(*node, *key),
            WarpOp::CreateEdge { id, .. } =>
                WarpOpKey::CreateEdge(*id),
            // ... etc
        }
    }
}
```

> **Watch Out:** The sort key intentionally *excludes* the operation's value. This means two `SetAttachment` ops to the same node/key will sort together, and only one survives deduplication.
>
> If they have *different* values, that's a conflict! The merge will keep the one with the lower `OpOrigin`, but this might indicate a footprint bug—two rewrites shouldn't write different values to the same location.

18.3 State Mutation Methods

File: `crates/warp-core/src/graph_store.rs`

After merging, the delta is applied to the graph:

```
apply_delta(delta: &TickDelta)
|
+-- FOR op IN &delta.ops:
    |
    +-- WarpOp::CreateNode(id) →
    |     self.nodes.insert(id, NodeRecord::default())
    |
    +-- WarpOp::DeleteNode(id) →
    |     self.nodes.remove(&id)
    |     // Cascades to edges referencing this node
    |
    +-- WarpOp::SetAttachment { node, key, value } →
    |     self.attachments.insert((node, key), value)
    |
    +-- ... (edges, etc.)
```

Application order matters for creates vs deletes vs updates. The sort key ensures:

1. Creates come before modifications to the created node
2. Deletes come after all modifications (so we don't update then immediately delete)
3. Within each category, operations are ordered by `OpOrigin`

This ordering is baked into the `WarpOpKey` enum's derived `Ord` implementation.

Pro Tip: If you're debugging unexpected state, dump the merged delta before application. You can see exactly which ops, in which order, will mutate the graph. The `OpOrigin` tells you which rule produced each op.

Chapter 19

Hash Computation

> **The Big Picture:** Hashes are the fingerprints of determinism. Three hashes define a commit: the `state_root` (what the graph looks like), the `patch_digest` (what operations produced it), and the `commit_hash` (the unique identity of this point in history).
>
> If any of these differ between runs, determinism is broken.

19.1 State Root

Entry Point: `compute_state_root()`
File: `crates/warp-core/src/hashing.rs:45-89`

```
compute_state_root(graph: &GraphStore) → Hash32
|
+-- let mut hasher = Blake3::new()
|
+-- // Hash nodes in canonical order
|   FOR (id, record) IN graph.nodes.iter() (sorted by id):
|       hasher.update(id.as_bytes())
|       hasher.update(&record.type_id.to_le_bytes())
|
+-- // Hash edges in canonical order
|   FOR (id, edge) IN graph.edges.iter() (sorted by id):
|       hasher.update(id.as_bytes())
|       hasher.update(edge.from.as_bytes())
|       hasher.update(edge.to.as_bytes())
|
+-- // Hash attachments in canonical order
|   FOR ((node, key), value) IN graph.attachments.iter():
|       hasher.update(node.as_bytes())
|       hasher.update(key.as_bytes())
|       hasher.update(&value.len().to_le_bytes())
|       hasher.update(value)
|
+-- hasher.finalize() → [u8; 32]
```

Why BLAKE3? It's fast (parallel by design), cryptographically secure, and has a simple API. The 32-byte output is enough for collision resistance while being compact.

Why sort everything? The graph's internal storage might use hash maps, which have non-deterministic iteration order. Sorting by ID ensures the same graph always produces the same hash, regardless of insertion order.

Why include lengths? Without length prefixes, `value = "ab"` followed by `next = "cd"` hashes the same as `value = "abcd"`. Length prefixes prevent this ambiguity.

19.2 Commit Hash v2

Entry Point: `compute_commit_hash_v2()`
File: `crates/warp-core/src/hashing.rs:120-156`

```rust
pub fn compute_commit_hash_v2(
    parents: &[Hash32],
    state_root: &Hash32,
    patch_digest: &Hash32,
    schema_hash: &Hash32,
    tick: u64,
    policy_hash: &Hash32,
) -> Hash32 {
    let mut hasher = Blake3::new();
    hasher.update(b"echo-commit-v2");  // Domain separator

    // Parents (sorted for determinism)
    let mut sorted_parents = parents.to_vec();
    sorted_parents.sort();
    hasher.update(&(sorted_parents.len() as u64).to_le_bytes());
    for p in &sorted_parents {
        hasher.update(p);
    }

    hasher.update(state_root);
    hasher.update(patch_digest);
    hasher.update(schema_hash);
    hasher.update(&tick.to_le_bytes());
    hasher.update(policy_hash);

    hasher.finalize().into()
}
```

Watch Out: The domain separator `b"echo-commit-v2"` is crucial. Without it, a commit hash could collide with a state root or patch digest that happens to have the same bytes. Domain separation is a cryptographic best practice.

The `v2` suffix leaves room for future commit hash formats while maintaining backward compatibility.

19.3 Patch Digest

Entry Point: `compute_patch_digest()`
File: `crates/warp-core/src/hashing.rs:95-115`

```
compute_patch_digest(delta: &TickDelta) → Hash32
|
+-- let mut hasher = Blake3::new()
+-- hasher.update(b"echo-patch-v1")
|
+-- FOR (op, origin) IN delta.ops.iter().zip(&delta.origins):
    |
    +-- hasher.update(&op.discriminant().to_le_bytes())
    |   The operation type (create/delete/set/etc.)
    |
    +-- op.hash_payload(&mut hasher)
    |   Operation-specific data
    |
    +-- hasher.update(&origin.to_bytes())
        Who produced this op
|
hasher.finalize()
```

> The patch digest captures *how* we got to the state, not just the final state. Two different sequences of operations might produce the same `state_root`, but they'll have different `patch_digest`s.
>
> This matters for auditing and replay: you can verify not just the result, but the exact sequence of operations that produced it.

> **Pro Tip:** When debugging determinism failures, compare patch digests first. If they differ but state roots match, you have operations happening in different orders. If state roots differ, the operations themselves are producing different effects.

Chapter 20

Commit Orchestration

Entry Point: `Engine::commit()`
File: `crates/warp-core/src/engine_impl.rs:785-892`

> **The Big Picture:** Commit is the grand finale of each tick. All the pieces come together: scheduled rewrites execute in parallel, deltas merge canonically, hashes are computed, and the worldline advances. This is where determinism is proven or broken.

20.1 Complete Call Trace

```
Engine::commit(tx: TxId) → Result<CommitResult, EngineError>
|
+-- // Phase 1: Drain and Schedule
|   let drained = self.scheduler.drain()
|   let admitted = self.scheduler.reserve(drained)
|
+-- // Phase 2: BOAW Parallel Execution
|   let view = self.graph.view()
|   let deltas = execute_parallel(view, &admitted, self.workers)
|
+-- // Phase 3: Merge Deltas
|   let merged = merge_deltas(deltas)
|
+-- // Phase 4: Apply to State
|   self.graph.apply_delta(&merged)
|
+-- // Phase 5: Compute Hashes
|   let state_root = compute_state_root(&self.graph)
|   let patch_digest = compute_patch_digest(&merged)
|   let commit_hash = compute_commit_hash_v2(
|       &self.parents,
|       &state_root,
|       &patch_digest,
|       &self.schema_hash,
|       self.tick,
|       &self.policy_hash,
```

```
|   )
|
+-- // Phase 6: Finalize
|   self.tick += 1
|   self.parents = vec![commit_hash]
|   self.live_txs.remove(&tx.value())
|
+-- RETURN CommitResult {
       commit_hash,
       state_root,
       patch_digest,
       ops_applied: merged.ops.len(),
   }
```

> Notice how the commit is structured as a pipeline. Each phase has a clear input and output:
>
> 1. Drain/Schedule: `PendingRewrite` → `ExecItem`
> 2. Execute: `ExecItem` → `Vec<TickDelta>`
> 3. Merge: `Vec<TickDelta>` → `TickDelta`
> 4. Apply: `TickDelta` → mutated graph
> 5. Hash: graph → `Hash32`s
>
> This clean separation makes the code testable. You can unit test each phase independently.

20.2 Commit Hash Inputs

The commit hash is a function of:

Input	Source
`parents`	Previous commit(s)—usually one, multiple for merges
`state_root`	Hash of the entire graph state
`patch_digest`	Hash of the operations applied
`schema_hash`	Hash of the type/rule schema
`tick`	Monotonic tick counter
`policy_hash`	Hash of active policies

> **Watch Out:** If you change the schema (add a rule, modify a type), the `schema_hash` changes, which changes all subsequent commit hashes. This is intentional—it prevents replaying a commit against an incompatible schema.
>
> When evolving your schema, consider migration strategies. Breaking changes should bump the schema version.

20.3 Error Handling

```rust
pub enum CommitError {
    NoActiveTransaction,
    SchedulerEmpty,       // Nothing to commit
    ExecutionFailed(String),
    HashMismatch {        // Determinism violation!
        expected: Hash32,
        actual: Hash32,
    },
}
```

Watch Out: `HashMismatch` is the "determinism alarm." If you ever see this error, something is seriously wrong—the same operations produced different results. Common causes:

- Floating-point operations in rules (use `F32Scalar`)
- HashMap iteration in footprint computation
- Time/random dependencies in rule logic
- Thread-local state leaking into rules

Echo includes a determinism fuzzer (`run_pair_determinism`) that runs commits twice with different schedulers and compares hashes.

Pro Tip: Enable the `delta_validate` feature during development. It records `OpOrigin` for every operation, making it easy to trace which rule produced unexpected state changes.

Chapter 21

Complete Call Graph

This chapter assembles the entire journey from user action to committed state. Every prior chapter focused on one subsystem; here we see them orchestrated.

21.1 Full Journey: Intent → Commit

```
USER ACTION
    |
    v
Engine::ingest_intent(intent_bytes)
    +- compute_intent_id()                // BLAKE3 content hash
    +- make_node_id(), make_type_id()     // Structural IDs
    +- store.insert_node()                // Create event node
    +- store.set_node_attachment()        // Attach intent payload
    +- store.insert_edge()                // Pending edge to inbox
    |
    v
Engine::begin() -> TxId
    +- tx_counter.wrapping_add(1)
    +- live_txs.insert(tx_counter)
    +- TxId::from_raw(tx_counter)
    |
    v
Engine::dispatch_next_intent(tx)          // (or manual apply)
    |
    v
Engine::apply(tx, rule_name, scope)
    +- Engine::apply_in_warp(tx, warp_id, rule_name, scope, &[])
        +- rules.get(rule_name)           // Lookup rule
        +- GraphView::new(store)          // Read-only view
        +- (rule.matcher)(view, scope)    // Match check
        +- scope_hash()                   // BLAKE3 ordering key
        +- (rule.compute_footprint)(view, scope)  // Footprint
        +- scheduler.enqueue(tx, PendingRewrite)
            +- PendingTx::enqueue()       // Last-wins dedup
    |
    v
Engine::commit_with_receipt(tx)
    |
    +--[DRAIN]
    |   scheduler.drain_for_tx(tx)
```

```
|       +- PendingTx::drain_in_order()
|           +- radix_sort() or sort_unstable_by()
|               20-pass LSD radix sort
|               ORDER: (scope_hash, rule_id, nonce)
|
+--[RESERVE]
|   FOR rewrite IN drained:
|       scheduler.reserve(tx, &mut rewrite)
|           +- has_conflict(active, pr)
|           |   +- GenSet::contains() x N    // O(1) per check
|           +- mark_all(active, pr)
|               +- GenSet::mark() x M        // O(1) per mark
|
+--[EXECUTE]
|   apply_reserved_rewrites(reserved, state_before)
|       FOR rewrite IN reserved:
|           (executor)(view, &scope, &mut delta)
|               +- scoped.emit(op)
|                   +- delta.emit_with_origin(op, origin)
|       delta.finalize()              // Sort ops
|       patch.apply_to_state(&mut self.state)
|
+--[MATERIALIZE]
|   bus.finalize()
|
+--[DELTA PATCH]
|   diff_state(&state_before, &self.state)
|       +- Sort by WarpOp::sort_key()
|   WarpTickPatchV1::new(...)
|       +- compute_patch_digest_v2()
|
+--[HASHES]
|   compute_state_root(&self.state, &self.current_root)
|       +- BFS reachability
|       +- BLAKE3 over canonical encoding
|   compute_commit_hash_v2(state_root, parents, ...)
|       +- BLAKE3(version || parents || state_root || ...)
|
+--[FINALIZE]
    CommitReceipt { hash, patch, state_root }
```

21.2 Complexity Summary

Phase	Complexity	Notes
Ingest	$O(1)$	Hash + insert
Begin/Abort	$O(1)$	Counter increment
Apply	$O(F)$	F = footprint size
Drain	$O(n \log n)$	Radix or comparison sort
Reserve	$O(n \cdot F)$	GenSet lookups
Execute	$O(n \cdot W)$	W = work per rule
Delta Merge	$O(n \log n)$	Dedup sort
State Root	$O(V + E)$	BFS traversal
Commit Hash	$O(1)$	Fixed-size BLAKE3

All operations are $O(1)$, $O(n)$, or $O(n \log n)$—nothing quadratic or exponential. The system scales linearly with work volume.

The one potential bottleneck is `compute_state_root` at $O(V + E)$, traversing the reachable graph. In practice, graphs are partitioned across warp instances, keeping each traversal manageable.

21.3 Determinism Boundaries

21.3.1 Guaranteed Deterministic

- Radix sort ordering (20-pass LSD)
- `BTreeMap`/`BTreeSet` iteration
- BLAKE3 hashing
- GenSet conflict detection
- Canonical merge deduplication

21.3.2 Intentionally Non-Deterministic

These are *handled by the canonical merge*:

- Worker execution order in BOAW
- Shard claim order (atomic counter)

Echo's determinism guarantee: *given the same inputs (intents, rules, initial state), the output (commit hash) is identical across all executions.*

This holds even though workers execute in arbitrary order, shards are claimed non-deterministically, and thread scheduling varies between runs. The canonical merge absorbs this chaos, producing deterministic output from non-deterministic intermediates.

21.3.3 Protocol Constants (Frozen)

- `NUM_SHARDS = 256`
- `SHARD_MASK = 255`
- Shard routing: `LE_u64(node_id[0..8]) & 255`
- Commit hash v2 version tag: `0x02 0x00`

These constants are "frozen"—changing them would break compatibility with existing commits. Protocol evolution happens through version tags, not by modifying existing constants.

21.4 End of Tour

We have seen:

Content-addressed everything From intents to commits, identity comes from content

Deterministic scheduling Radix sort + footprints = predictable execution

Safe parallelism Sharded execution + canonical merge = speed without chaos

Cryptographic integrity BLAKE3 hashes throughout = verifiable state

Echo is built from simple, composable primitives—content hashing, sorted iteration, conflict-free sharding—assembled into a system that solves hard distributed problems. The code rewards careful study.

Happy hacking!

License and Legal Notice

This project is made available under an open source, dual-licensing model.

Code

All *code* in this repository—including Rust source files, scripts, build tooling, and any compiled binaries—is licensed under the **Apache License, Version 2.0**.

- Canonical text: `LICENSE-APACHE`
- SPDX identifier: `Apache-2.0`

Users may use, modify, and redistribute the code under the terms of the Apache License, Version 2.0.

Theory, Mathematics, and Documentation

The *theory*, *mathematics*, and *documentation* corpus associated with this project—for example LaTeX sources, notes, and expository materials—is dual-licensed under:

1. the Apache License, Version 2.0 (`Apache-2.0`), *or*
2. the MIND-UCAL License, Version 1.0 (`MIND-UCAL-1.0`),

at the user's option.

If you do not wish to use MIND-UCAL, you may freely use all theory, mathematics, and documentation under the Apache License, Version 2.0 alone. No part of this project requires adopting MIND-UCAL in order to be usable.

SPDX Headers

To make licensing machine-readable and unambiguous, the project uses SPDX license identifiers in file headers. Typical examples include:

- Code files (Rust, scripts, etc.):

  ```
  // SPDX-License-Identifier: Apache-2.0
  ```

- Documentation and theory files (Markdown, LaTeX, etc.):

  ```
  % SPDX-License-Identifier: Apache-2.0 OR MIND-UCAL-1.0
  ```

These identifiers correspond directly to the licenses described above.

Disclaimer

Unless required by applicable law or agreed to in writing, the material in this project is provided on an "AS IS" basis, without warranties or conditions of any kind, either express or implied. For the full terms, see `LICENSE-APACHE` and `LICENSE-MIND-UCAL`.